

Agroverse Chocolate Subscriptions — pre-flight + execution roadmap

For: TrueSight DAO / Agroverse contributors
From: Claude (agentic AI) — design ratified with Gary Teh
Date: 2026-06-09
Status: Roadmap aligned · implementation begins at Phase 1 PR1.1

Goal: Let supporters subscribe to a recurring monthly shipment of Agroverse ceremonial-cacao **chocolate bars** (generic, vintage-independent), pick the quantity themselves, manage/cancel without operator involvement, and have fulfillment land with **near-zero manual relay** between Gary and Kirsten. First real subscriber: **Linda (Rochester, NY)**. The self-serve subscribe page is also intended as a **cacao-circle event placard QR** target.

► RESUME HERE

► **ACTIVE: Phase 1, PR1.1 — generic SKU + subscription schema** in `products.js` (`agroverse_shop_beta`). Then PR1.2 (shared subscribe engine) → PR1.3 (the `/subscribe/chocolate-bar/` clean-URL wrapper) → PR1.4 (GAS action) → PR1.5 (generic-bar PDP). Architecture is **one data-driven engine, clean path URLs, catalog-as-source-of-truth** — see *Decisions + Generic SKU model + PDP spec*. Nothing implemented yet (doc written 2026-06-09, architecture folded in 2026-06-09). Start at the **Pre-flight checklist** (GAS deploy path + generic SKU definition), then PR1.1. Each PR is independently shippable; after any PR merges, report the DAO contribution before starting the next (see `DAO_CLIENT_AI_AGENT_CONTRIBUTIONS.md`).

Companion docs (read before touching the relevant layer): - `STRIPE_LEDGER_ROUTING.md` — all 5 Stripe flows; the `[LEDGER_ID]` routing pattern; the Stripe Social Media Checkout ID audit tab. - `notes/claude_serialized_qr_sales_2026-04-29.md` — the one-`[SALES_EVENT]`-per-QR-code pattern + fee amortization (the fulfillment fan-out reuses this verbatim). - `EDGAR_DAO_EXTRACTION_PLAN.md` — Edgar→`dao_protocol` split. **The Stripe webhook ENTRY (`/stripe_webhook`) intentionally stays on Rails (`sentiment_importer`)**; new DAO-glue read endpoints go to `dao_protocol (:8010)`. - `DAPP_PAGE_CONVENTIONS.md` + `dapp/UX_CONVENTIONS.md` — for the dapp fulfillment page (Phase 2) and the `agroverse_shop` sign-in (Phase 3).

Design summary (the one insight)

The current one-off flow already records the **physical reality after the fact**: Kirsten picks whatever bars are in stock, then the actual QR codes that shipped are recorded via `report_sales.html`. The clumsiness is the human *relay*, not the model. So:

- **Never pre-assign QR codes.** A subscription is a *recurring charge + a monthly fulfillment obligation*. The specific QR codes are bound at **pack time** by Kirsten, in one batch action. This dissolves the “what if Kirsten can’t find that exact bar” problem — she grabs any N in-stock bars and records what she grabbed.

- **Generic, vintage-independent SKU.** Decouples the subscription commitment from any single harvest. **Traceability is preserved at the unit level** — every bar still carries its QR → vintage → farm; the subscriber *discovers* provenance by scanning (the Cacao Chasers surprise model), they just don't pre-select.
- **Fulfillment = N existing [SALES EVENT] s**, not a new event type. The entire QR→ledger→treasury chain runs unchanged; revenue auto-routes to each bar's own AGL ledger via its `ledger_shortcut`. Only genuinely new pieces: the obligation **queue** and a **batch-submit UI** that loops (≈ `dao_client/examples/bulk_qr_sales_template.py`).
- **Identity = verified email, server is source of truth.** Reuse the canonical RSA flow (capoeira/oracle/dapp). Cross-browser portability is solved by making the verified email the key and reading orders/subscriptions **server-side** — `localStorage (agroverse_order_history)` becomes a cache, never synced between browsers. Subscription management = a **Stripe Customer Portal** link minted for the matching Customer (works from any browser).

Decisions (ratified with Gary, 2026-06-09)

#	Decision
Scope	Chocolate bars only for now (no generic ceremonial-cacao SKU yet).
Billing engine	Stays in the existing GAS (<code>agroverse_shop_checkout.gs</code> → <code>createCheckoutSession</code>). Subscriptions reuse the same dynamic inline price_data , adding <code>recurring:{interval:'month'} + mode:'subscription'</code> . No pre-created Stripe Price IDs. Confirmed: Stripe Checkout supports on-the-fly recurring <code>price_data</code> .
Quantity	User-chosen but BOUNDED — preset chips (3 / 6 / 12) + stepper, default 6, hard cap ~24. Not an unbounded field: above the cap, route to wholesale (“buying for your shop?”). Rationale: per-bar manual fulfillment + shipping locked at signup + typo guard.
One-off vs subscription	Both — Subscribe is the hero CTA, one-off Add-to-Cart is the secondary on-ramp. The “don't hold generic one-off stock” worry is moot : all bars share one GTIN, so one-off generic sells from the <i>same</i> pool as the vintage PDPs (fulfilled exactly like today).
GTIN model	One GTIN per product TYPE, not per vintage. All chocolate bars share a single chocolate-bar GTIN; all ceremonial cacao share a single cacao GTIN. The QR code (not the GTIN) differentiates farm/vintage. So the generic SKU reuses the existing shared GTIN — do NOT mint a new one. The vintage PDPs are presentation sub-views of the same GTIN/product.
Pricing	Same basis as the shop today — generic bar unit price × quantity (dynamic), computed server-side.
Shipping	Computed from signup address — run EasyPost once at signup, lock that amount as a recurring shipping line item (subscription mode has no interactive <code>shipping_options</code> picker, so a recurring line is the correct mechanism).
Fulfillment attribution	On each per-bar [SALES EVENT] : Sold by = Kirsten Ritschel, Cash proceeds collected by = Gary Teh (same as today).
Start order	Phase 1 (the subscribe engine) first. But Linda goes live only after Phase 2 (fulfillment queue + <code>invoice.paid</code> handler) or the interim bridge — see the <i>Activation gate</i> . Phase 1 ends at a promoted-but-not-yet-advertised page, not at “send Linda the link.”
Identity	Adopt the canonical RSA email-bound flow on <code>agroverse_shop</code> (Phase 3); server-side read-by-email retires the <code>localStorage</code> -only history limitation.

#	Decision
Beta-first	All UI work lands in <code>agroverse_shop_beta</code> / <code>dapp_beta</code> ; human promotes to prod. Never push prod directly.
Page architecture	One data-driven subscribe ENGINE, not a page per product. A single <code>/subscribe/</code> engine renders entirely from the catalog entry; each subscribable SKU is exposed via a thin clean-URL wrapper — <code>/subscribe/chocolate-bar/</code> now, <code>/subscribe/ceremonial-cacao/</code> later. Adding a new generic line = a catalog entry + a ~10-line wrapper, no new engine, GAS action, or fulfillment code . Clean path URLs (not <code>?sku=</code>) for SEO / sharing / placard QR.
Generic SKU model	Each generic, vintage-independent line = one catalog entry per GTIN in <code>products.js</code> , carrying subscription metadata (<code>subscribable</code> , <code>cadence</code> , <code>min/max/defaultQty</code>). The SAME entry feeds two render surfaces : the existing PDP system (one-off buy + Subscribe CTA) and the shared subscribe engine . Origin is <code>rotating</code> (not one farm/shipment); provenance is per-bar via the QR.
Stripe topology	Session creation = the Apps Script (cart/subscribe flow, always GAS). Webhook = Rails <code>sentiment_importer</code> / <code>stripe_webhook</code> — it stayed on Rails (extraction-plan decision A; shared with Edgar trading-SaaS subs). dao_protocol is NOT in the subscription path . So <code>invoice.paid</code> handling (PR2.2) is Rails, and sandbox-testing uses local Rails + Stripe CLI (see <i>Sandbox / test setup</i>).
Cancel path	Phase 1: Stripe no-code Customer Portal login link (dashboard config, email-based, zero build) — so “cancel/modify anytime” is true the day Linda goes live. Phase 3 replaces it with the in-app RSA-gated portal (PR3.4). Don't promise self-serve cancel without one of these wired.

Pre-flight checklist (verify BEFORE writing code)

- **[] GAS deploy path for the checkout script.** Confirm which clasp mirror deploys `agroverse_shop/google-app-script/agroverse_shop_checkout/agroverse_shop_checkout.gs` (the in-repo copy is reference). Capture the scriptId + the existing `/exec` deployment URL so subscription sessions deploy without breaking the cart. **⚠ GAS is NOT beta-isolated:** the Apps Script backend is ONE deployment shared by beta AND prod sites — deploying hits prod immediately; the beta-first rule does not apply to GAS. Mitigation: make the new action **purely additive** (new function; existing cart `createCheckoutSession` untouched) so it's inert until the beta `/subscribe/` page calls it. **If the script is NOT a tokenomics clasp mirror, Sophia's gas_deploy_project tool (targets context/tokenomics) cannot deploy it — flag to operator.**
- **[] Generic SKU definition (GTIN/price/images are in-repo — no operator ask).** GTIN = `00860010660256` (shared 81% 50g bar GTIN; reuse, never mint); **price + images** already in `agroverse_shop/js/products.js` / `assets/images/products/` (reuse the bar + packaging-back-QR shots). Keep the `Agroverse SKUs` sheet (col J = GTIN) in sync. **Operator-gated = copy sign-off only:** the generic **name/positioning** (proposed: `Ceremonial Cacao Chocolate Bar — Single-Estate, Rotating Origins`). (*Decided: keep one-off + subscribe; quantity bounded with wholesale overflow.*)
- **[] Stripe sandbox — HARD GATE (blocks PR1.7).** See *Sandbox / test setup* below. Two legs: (a) the **GAS** needs a `STRIPE_TEST_SECRET_KEY` Script Property + `environment=development` switch (PR6a's `sk_test` was Edgar's `qr-code-check`, a different codebase — does **not** cover the checkout GAS); (b) the **webhook stays on Rails** — test with local `sentiment_importer` + **Stripe CLI** `listen/trigger`. **If the GAS test switch doesn't exist, STOP — do not smoke-test with a real card.**
- **[] Webhook coverage.** Confirm `sentiment_importer/app/controllers/webhook_controller.rb#stripe` currently handles only `checkout.session.completed`; Phase 2 adds `invoice.paid` (recurring renewals). Note: the **first charge** arrives as `checkout.session.completed` (mode=subscription); **renewals** arrive as `invoice.paid` — both must create a fulfillment obligation.

- [] **Downstream session-id prefix.** The sales parser validates `cs_(live|test)_` prefixes and strips `(none)`. Subscription renewals carry `in_.../sub_...`, not `cs_...`. Decide: teach the parser the new prefix, or stash the invoice id in a separate attr and leave “Stripe Session ID” blank. (Phase 2, PR2.4.)
- [] **Sheet writes.** Confirm the service account that may write the new “Subscription Fulfillment Queue” tab (likely `agroverse-qr-code-manager@...`); `Currencies` stays range-protected (not touched here).
- [] **dao_client auth active** for contribution reporting (`auth.py status`).

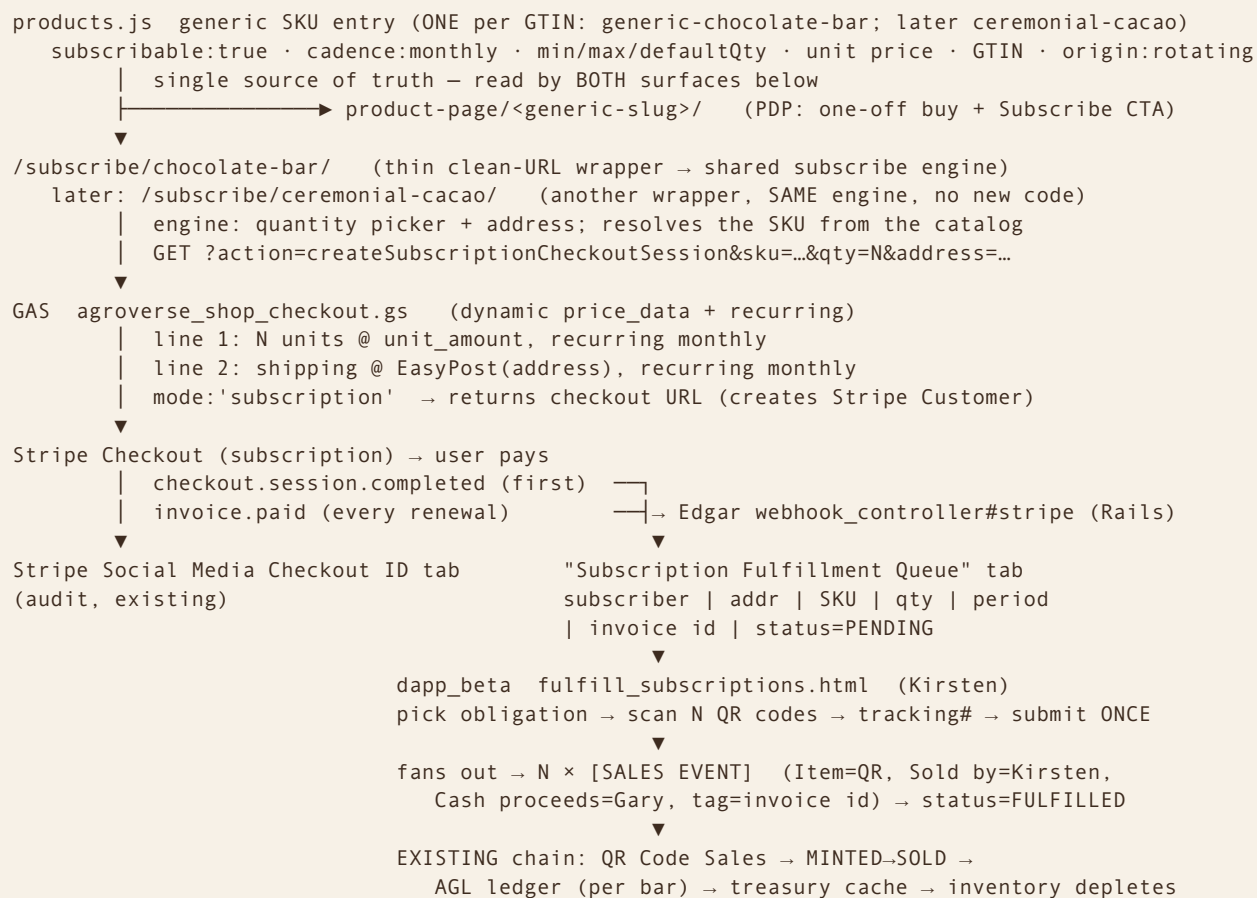
Sandbox / test setup (how to test without real charges)

Topology recap: session creation = **GAS**; webhook (first charge + `invoice.paid` renewals) = **Rails**
`sentiment_importer` / `stripe_webhook` (it stayed on Rails — *not* `dao_protocol`). So testing has two independent legs:

1. **Stripe Test mode** (or a Stripe **Sandbox** — newer isolated env). Gives `sk_test` / `pk_test` + test cards (4242 4242 4242 4242).
2. **GAS leg (session creation):** add a `STRIPE_TEST_SECRET_KEY` Script Property and an `environment=development` param to `createSubscriptionCheckoutSession` — mirror the env-switch `createLedgerCheckoutSession` already does (`STRIPE_LEDGER_ROUTING.md`). The beta `/subscribe/` page sends `environment=development` from localhost. Test by completing a test-card subscription → a real test-mode subscription + invoices appear in the Stripe **test** dashboard.
3. **Webhook leg (Rails — your existing method still works):** run `sentiment_importer` locally and use the **Stripe CLI**:
4. `stripe listen --forward-to localhost:<railsport>/stripe_webhook`
5. `stripe trigger checkout.session.completed` (first charge) and `stripe trigger invoice.payment_succeeded` (renewal) — simulate a month’s renewal **instantly**, no waiting a cycle. `dao_protocol` need not be running; Rails’ delegate POST to `:8010/stripe/order_sync` fail-softs locally (rescued).

Do NOT “smoke test” by paying with a real card on prod keys. If the GAS test switch (step 2) isn’t in place yet, that’s a STOP — build it first (pre-flight).

Architecture (where each piece lives)



Generic SKU model + PDP spec

Catalog entry (products.js) — one per generic GTIN

The generic, vintage-independent line is a normal catalog row plus subscription metadata. It is the **single source of truth** for both the PDP and the subscribe engine. Proposed shape (extends today's fields: `productId`, `productPageSlug`, `name`, `price`, `weight`, `image`, `category`):

```
{
  productId: 'generic-ceremonial-cacao-chocolate-bar',
  productPageSlug: 'ceremonial-cacao-chocolate-bar',    // PDP at /product-page/<slug>/
  name: 'Ceremonial Cacao Chocolate Bar – Single-Estate, Rotating Origins',
  gtin: '00860010660256',                                // the shared 81% 50g bar GTIN (already in products.js)
  price: 10.00,                                           // unit price (confirm vs current bar price)
  weight: 1.76,                                           // oz, per bar (50g)
  image: '/assets/images/products/generic-bar-hero.jpg',
  category: 'retail',
  origin: 'rotating',                                    // sentinel – NOT one farm/shipment
  // subscription metadata (new):
  subscribable: true,
  subscriptionSlug: 'chocolate-bar',                     // → /subscribe/chocolate-bar/
  cadence: 'monthly',
  minQty: 1, maxQty: 24, defaultQty: 6,                  // Linda = 6/month; presets 3/6/12 + stepper; >cap → w
}
```

Adding ceremonial cacao later = **another entry** (`subscriptionSlug: 'ceremonial-cacao'`) + its wrapper + its PDP. Engine, GAS action, fulfillment queue: unchanged.

Generic-bar PDP — what goes on the page

The generic PDP is both a discoverable product page (one-off purchase) and the marketing surface that drives the subscription. It diverges from a normal single-estate PDP in one key way: **it is not bound to a single farm/shipment**, so the “Products from This Farm / Shipment” cross-listing grids in `AGROVERSE_SHOP_NEW_SKU_WEB_CHECKLIST.md` do **not** apply. Instead it links out to “the farms we rotate through.”

Imagery (follow the PDP image convention — hero at full width, supplementary in a `.gallery` grid inside `.product-image-container`): - **Hero = the product**, not a farm portrait (there’s no single farm) — the bar + packaging front. - **Gallery**: bar close-up; **packaging back showing the traceability QR** (this is the hook — it’s how you learn the origin); optionally a montage/map of the rotating Bahia origins.

Content blocks (top → bottom): 1. **Title + the concept.** Lead copy explains the model: every bar is a single estate from a **rotating** Bahia farm; you **discover the exact farm + vintage by scanning the QR on the back**. Frame as a feature — “a monthly journey through our farms” (the Cacao Chasers surprise model) — not a limitation. 2. **Primary CTA → Subscribe.** “Subscribe — N bars/month” linking to `/subscribe/chocolate-bar/`; note **cancel/modify anytime** (Stripe Customer Portal, Phase 3). Show the default 6/month, adjustable. 3. **Secondary CTA → one-off.** Standard Add-to-Cart for a single generic bar (sells from the same shared-GTIN pool as the vintage PDPs). **Decided: keep one-off** — it’s a free on-ramp with no extra stock to hold. 4. **What you get.** 6 bars/month (configurable), shipped from SF, each QR-traceable to its estate + vintage. 5. **Provenance & traceability.** Every bar carries a QR → exact farm, vintage, on-chain ledger entry; link to the scanner / “how it works.” 6. **Tasting & spec.** House style for the rotating line (e.g. 81% dark, ceremonial-grade, minimal ingredients), ingredients, allergens, **per-bar weight (50g)**. 7. **Shipping & subscription terms.** Ships monthly; shipping computed at signup and locked; manage via the portal. 8. **Wholesale banner** between the CTA block and the description, per the SKU checklist convention (`../../wholesale/`). 9. **Impact.** Regenerative, single-estate, supports farmers + the DAO.

Feeds / Merchant Center / JSON-LD: the **one-off** generic bar is a normal feed product using the **existing shared chocolate-bar GTIN** (do not mint a new one; all bars share it). Heads-up: the multiple vintage PDPs already share that one GTIN, so watch for Merchant Center duplicate-GTIN warnings — the generic PDP is arguably the canonical one. Follow `agroverse_shop/docs/PRODUCT_CREATION_CHECKLIST.md`. The **subscription** itself is a site feature, not a Merchant Center product — don’t model the recurring SKU in the product feed.

Execution roadmap (resume tracker)

Legend: ☐ todo · **X** in progress · ☒ merged · 🍌 DAO contribution reported

Phase 1 — Generic SKU + data-driven subscribe engine (START HERE)

Architecture: one shared engine, clean path URLs, catalog-as-source-of-truth (see *Generic SKU model + PDP spec* and the *Decisions* table). Build the engine once; expose each line via a thin wrapper.

PR	Repo	Scope	Status
1.1	<code>agroverse_shop_beta</code>	Generic SKU + schema. Add the subscription metadata fields to <code>products.js</code> (<code>subscribable</code> , <code>subscriptionSlug</code> , <code>cadence</code> , <code>min/max/defaultQty</code> , <code>gtin</code> , <code>origin: 'rotating'</code>) and the first generic-bar catalog entry (price, weight, hero image). Source of truth for all downstream.	☐

PR	Repo	Scope	Status
1.2	agroverse_shop_beta	Shared subscribe engine (<code>/subscribe/</code>): resolves the SKU from the catalog, renders quantity picker (min/max/default) + address form (reuse <code>checkout-form-storage.js</code>), calls the GAS action, redirects to Stripe. No product-specific code — fully data-driven.	☐
1.3	agroverse_shop_beta	Clean path wrapper <code>/subscribe/chocolate-bar/</code> — ~10-line page that points the engine at <code>subscriptionSlug: 'chocolate-bar'</code> . This is the placard-QR + Linda URL. (Template for future <code>/subscribe/ceremonial-cacao/</code> .)	☐
1.4	GAS (checkout)	New ADDITIVE <code>createSubscriptionCheckoutSession</code> action — do NOT touch the existing cart <code>createCheckoutSession</code> (GAS is shared beta+prod; additive keeps it inert until the beta page calls it). Unit line <code>recurring: { interval: 'month' }</code> ; run existing EasyPost weight+rate code once → recurring shipping line (one tier , e.g. Ground Advantage); <code>mode: 'subscription'</code> ; <code>environment</code> test/live key switch (see <i>Sandbox</i>); return <code>checkoutUrl</code> ; key off <code>sku</code> .	☐
1.5	agroverse_shop_beta	Generic-bar PDP at <code>/product-page/<generic-slug>/</code> per the <i>PDP spec</i> : discovery/rotating-origin copy, hero=product, gallery incl. packaging-back QR shot, primary Subscribe CTA → <code>/subscribe/chocolate-bar/</code> , optional one-off Add-to-Cart, provenance block, wholesale banner. (Cross-listing grids N/A — generic is not farm/shipment-bound.)	☐
1.6	sentiment_importer (Rails) / verify	Subscription-mode <code>checkout.session.completed</code> must not break <code>MetaCheckoutOrderSync</code> (it assumes <code>channel=='meta'</code> + <code>wix_products</code> — a sub session has neither → make it no-op/log cleanly, not error). Renewals arrive as <code>invoice.paid</code> , which nothing handles until PR2.2 — so until Phase 2, only the FIRST charge is recorded anywhere.	☐
1.7	promote (human)	Sandbox end-to-end smoke (Stripe test mode, GAS test key, Stripe-CLI <code>trigger</code>); beta → prod promotion (human). Do NOT onboard Linda here — see the Activation gate below.	☐

Adding ceremonial cacao later = data only: one new `products.js` entry

(`subscriptionSlug: 'ceremonial-cacao'`) + a `/subscribe/ceremonial-cacao/` wrapper + its PDP. The engine, GAS action, webhook, and Phase-2 fulfillment queue do **not** change.

⚠ **Activation gate (Linda / any real subscriber):** do NOT send the prod `/subscribe/chocolate-bar/` URL to a real person until **Phase 2** (queue + fulfillment UI + `invoice.paid` handler) is live — **OR** the interim bridge is in place. Otherwise monthly renewals are charged but **unrecorded & unfulfilled** (no `invoice.paid` handler until PR2.2; see PR1.6).

Interim bridge (only if activating before Phase 2): (1) watch renewals in the Stripe dashboard (or a throwaway `invoice.paid` → Telegram/sheet logger); (2) fulfill each renewal manually via `dao_client/examples/bulk_qr_sales_template.py` (one `[SALES EVENT]` /bar, Sold by=Kirsten / Cash=Gary, tag the invoice id); (3) give Linda the Stripe **no-code Customer Portal** link for cancel/modify.

Phase 2 — Fulfillment automation (kill the relay)

PR	Repo	Scope	Status
2.1	Google Sheets	Create “ Subscription Fulfillment Queue ” tab + schema (subscriber, email, address, SKU, qty, period, invoice id, status, fulfilled-by, tracking#, fulfilled-at).	☐
2.2	<code>sentiment_importer</code> (Rails)	In <code>webhook_controller#stripe</code> (the webhook stays on Rails): handle <code>invoice.paid</code> and the first subscription <code>checkout.session.completed</code> → append a PENDING obligation (new <code>Gdrive::*</code> writer), idempotent on invoice id. Lifecycle (so bad obligations aren’t created): <code>invoice.payment_failed</code> (dunning → NO obligation), <code>customer.subscription.deleted</code> (stop future obligations); refunds reverse/flag the obligation + its <code>[SALES EVENT]</code> s (v1 may do this manually — state which).	☐
2.3	<code>dapp_beta</code>	<code>fulfill_subscriptions.html</code> : list PENDING obligations, pick one, scan/enter N QR codes (reuse <code>report_sales.html</code> + <code>edgar_payload_helper.js</code>), enter tracking#, submit once → loop N <code>[SALES EVENT]</code> s (Sold by=Kirsten, Cash proceeds=Gary, tag invoice id) → mark FULFILLED.	☐
2.4	GAS (sales parser)	Accept subscription invoice/ <code>sub_</code> ids (or read a dedicated attr) so renewal-sourced <code>[SALES EVENT]</code> s reconcile without the <code>cs_</code> guard tripping.	☐
2.5	<code>dapp_beta</code> (stretch)	Low-stock alert on the queue: warn when generic-pool stock < upcoming obligations (supply-buffer second-order effect).	☐

Phase 3 — RSA accounts + cross-browser portability

PR	Repo	Scope	Status
3.1	<code>agroverse_shop_beta</code>	“Sign in” in nav: reuse canonical RSA flow (<code>publicKey/privateKey</code> <code>localStorage</code> , <code>edgar_payload_helper.js</code> , <code>[EMAIL REGISTERED EVENT]</code> →link→ <code>[EMAIL VERIFICATION EVENT]</code>).	☐
3.2	Google Sheets / Rails	Add buyer email column to the Stripe Social Media Checkout ID tab (write it on <code>append_record</code>); backfill from Stripe where feasible.	☐
3.3	<code>dao_protocol</code>	Read-by-verified-email endpoint → returns the email’s orders (Stripe tab) + active subscriptions (Stripe API).	☐
3.4	<code>dao_protocol</code>	Mint a Stripe Customer Portal session for the verified email’s Customer; “Manage subscription” button on the shop.	☐
3.5	<code>agroverse_shop_beta</code>	<code>/order-history/</code> reads the server endpoint when signed in (<code>localStorage</code> = fallback cache); one-time migration of local <code>agroverse_order_history</code> on first verify.	☐

Risks / open items

- **Supply buffer.** Subscriptions create a recurring inventory commitment; the generic pool must be replenished as vintages finish. PR2.5 surfaces it; supply planning is out of scope for code but in scope for Dr-Manhattan-level attention.

- **Shipping drift.** Locked-at-signup shipping can diverge from real cost over time (carrier increases, address-region changes). Acceptable for v1; revisit if margins slip. Customer Portal lets subscribers update address — decide whether an address change should re-quote shipping (likely a manual re-quote at first).
- **No double-count (RULE, not just timing).** The subscription Stripe charge is **audit-only** (Stripe Social Media Checkout ID tab); the per-bar `[SALES_EVENT]`s at fulfillment are the **sole revenue source of truth**. `stripe_sales_sync.gs` must **not** recognize the subscription charge as revenue — it has no `[LEDGER_ID]` prefix and is not a `TARGET_PRODUCT_ID`, so it's ignored today; **confirm that holds for subscription invoices**, and that the per-bar events reconcile against the invoice id (PR2.4). Otherwise the same money is counted twice. The charge↔fulfillment gap within a cycle is expected (the queue is the deferred-revenue tracker).
- **Single-vs-multi service shipping.** EasyPost returns multiple USPS rates; v1 picks one tier (e.g. Ground Advantage) computed for the address rather than letting the subscriber choose. Confirm the tier in PR1.2.

Plan owner: this doc. Update the resume tracker as each PR lands; report the DAO contribution before starting the next PR.